PrivFuse: Enforcing Privacy SLOs at Admission Time in Multi-Tenant Data Markets

Anonymous Submission
Paper ID: xxxx

***Abstract*—Data trading platforms sell answers computed over personal records contributed by many subjects, and they serve many tenants concurrently. Each subject is promised a bound on cumulative differential-privacy loss, but the promise is enforced today by accounting after the fact: budgets are tallied offline, keyed per tenant, and reconciled in batches. A tenant that splits its traffic across accounts or platforms therefore spends the same subject's budget many times over, and a crash between deciding and recording a query destroys the accounting entirely. We argue that privacy budgets should be enforced at admission, like any other service-level objective, and present PrivFuse, a runtime enforcement layer for multi-tenant data markets. PrivFuse rests on three ideas. First, admission control needs only a *one-sided* estimate of a subject's spend, so an exact per-subject counter can be replaced by a conservative Count-Min sketch plus a bounded exact table for heavy hitters, giving estimator memory independent of the subject population together with a guarantee that spend is never underestimated. Second, budget accounting is a transaction rather than an increment, so reservation, commit and release are separated behind a subject-striped coordinator that makes splitting and crash-restart attacks unprofitable and settlement exactly-once. Third, windowed SLOs with a hysteretic breaker turn a cumulative bound into predictable degradation. On traces with 10^6 subjects, PrivFuse bounds memory at 4.7 MB where exact accounting needs 265 MB, admits 0.93 of requests at a cap of ε=1.0 with zero violations where exact per-tenant blocking admits 0.98 with 12.7 violations, and holds the escape ratio at 0.82 while audit-only policies reach 2.43.**

***Index Terms*—differential privacy, data markets, admission control, privacy accounting, service-level objectives, count-min sketch**

I. Introduction

A data trading platform does not sell records. It sells answers: counts, summaries and trained statistics computed over personal data that many subjects contributed and that many tenants query concurrently. Blockchain-backed designs have made the settlement half of this exchange almost routine, with smart contracts escrowing payment and releasing keys against delivery [1]. The privacy half has not kept pace. A platform can prove that a tenant paid, and that a delivered object matches a committed digest, but it cannot say whether answering one more query would push some subject past the privacy budget the platform promised when the data was collected. That question is not about a single query; it is about the accumulated transcript of every query that touched the subject, from every tenant, on every platform of the market.

Differential privacy (DP) supplies the language for exactly this kind of statement [2], [3]. Its guarantee is compositional: the loss of a sequence of mechanisms is a function of the individual losses, so an operator who charges each release against a per-subject budget can bound the leakage of the transcript [4], [5]. The governance literature has largely taken the budget for granted, treating it as a parameter of policy rather than a running quantity that must be measured, reserved and settled [6], [7]. Practice has followed suit: budgets are compiled into policy documents, tallied in offline jobs, and reconciled at the end of an accounting period. Auditing tools then estimate whether the released artifacts respected the promise, either by attacking the mechanism [8], [9] or by comparing provable and empirical bounds [10], and the governance literature supplies the organisational scaffolding for such post-market [11] and ethics-based [12] review.

Retrospective accounting is defensible when a single curator releases a fixed set of statistics. It fails in a market, for three reasons that compound. First, the accounting unit is wrong. Budgets are keyed by tenant because the tenant pays, but the loss accrues to the subject, and a tenant that can open accounts at several platforms—or simply at several tenants of one platform—converts one budget into many. Second, the decision point is wrong. By the time an audit reports an overspend the answers have been released and cannot be recalled. Third, the bookkeeping is not a counter but a transaction. A crash between admission and settlement either loses the charge, so the next query is admitted against a budget that was already consumed, or duplicates it, so availability collapses for no reason. Our measurements make the first failure concrete: a tenant that splits a budget across 42 accounts achieves an escape ratio of 2.43 against audit-only enforcement and 1.69 against exact per-tenant blocking, and neither number is visible to the platform until after the fact.

The obvious repair—keep an exact counter for every subject—does not survive scale. At 10^6 subjects the counter table alone occupies 265 MB, it must be consulted on the critical path of admission, and it must be updated atomically, which serialises the coordinator. We take a different route, and it rests on one observation: admission control does not need to know a subject's spend, only whether it has crossed a threshold. That predicate tolerates error in one direction and not the other. An over-estimate costs availability, because a safe query is refused; an under-estimate costs soundness, because a cap is breached. If the estimator is one-sided—never below the truth—the platform can trade availability for memory without trading away the guarantee.

PrivFuse turns this into a runtime enforcement layer. A conservative Count-Min sketch over (subject, cost) keys answers threshold queries in constant memory and is one-sided by

construction, because its update rule raises only the cells that fall below the running minimum. A bounded exact table holds the heavy hitters, whose budgets are the ones that bind, and a subject returning after eviction is seeded with the sketch's upper bound for its past so that the exact path never starts below the sketch. A windowed SLO evaluator and a hysteretic breaker turn a cumulative bound into a predictable degradation curve, and a two-phase settlement protocol with a subject-striped coordinator makes admission atomic, idempotent, and robust to a crash between reserve and commit. The composition algebra is zero-concentrated DP, chosen not for tightness but because it is closed-form and invertible: admission must price a query and calibrate a target budget in microseconds, and the discrete order grid of Rényi DP cannot calibrate small targets at all (Section VII-G).

This paper makes four contributions.

1) We formulate privacy enforcement in a data market as an admission-control problem with service-level semantics: a per-subject cap over a sliding window, a tenant model in which the adversary is the splitter, and four objectives (soundness, availability, stability, atomicity) that must hold together (Section III).
2) We give a one-sided sketch accountant whose estimator memory is independent of the subject population, prove that it never underestimates spend, and show that hot-table seeding preserves one-sidedness across eviction (Section IV-C, Proposition 1).
3) We recast budget accounting as a transaction and give a striped coordinator with reserve/commit/release, establishing exactly-once settlement and single-spend safety under concurrent and crashing clients (Section IV-E, Propositions 2 and 3).
4) We implement PrivFuse as a reference system and evaluate it on twelve experiments over three seeds, including an ablation, a sensitivity sweep and a scaling study to 10^6 subjects (Section VII).

The measurements separate what the design buys from what it costs. At a cap of ε=1.0, PrivFuse admits 0.933 of requests with zero hard-cap violations, while exact per-tenant blocking admits 0.978 and violates the cap 12.7 times per run; the 4.5-point availability gap is the price of soundness. Estimator memory falls from 265 MB to 4.7 MB at 10^6 subjects, and the sketch path costs 32 μs per request against 4.6 μs for an exact table, a gap that is invisible next to query execution. The escape ratio stays at 0.82 however finely the attacker splits, because splitting does not change the subject key, while audit-only enforcement rises to 2.43. We also report where the design does not help: striping brings no throughput on two cores, and the SLO monitor loses coverage when the hot table is small.

II. Background and Motivation

A. Differential Privacy and Its Accounting

A randomized mechanism M satisfies (ε, δ)-differential privacy if for every pair of neighbouring datasets D, D' and every event S, $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta$. The definition is per-release, but its power comes from composition: the guarantee for a transcript follows from the guarantees of its parts. That is what lets a platform promise a subject a bound on *total* leakage rather than on each answer, and it is why the promise needs bookkeeping. Whatever its interpretation—Bayesian [13] or one of the variants surveyed in [3]—the quantity to bound is cumulative.

Composition bounds differ in how tightly they track it. Rényi DP [4] gives sharp constants for a fixed number of adaptively chosen mechanisms, at the cost of a discrete order parameter: the accountant evaluates a grid of orders and converts the result to (ε, δ). Zero-concentrated DP [5] bounds the Rényi divergence of order $1 + \alpha$ by $\rho\alpha$ for all $\alpha > 1$, composes by adding ρ, and converts in closed form,

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{1}$$

with inverse $\rho = \left(\sqrt{\ln(1/\delta) + \varepsilon} - \sqrt{\ln(1/\delta)}\right)^2$. A Gaussian mechanism with ℓ_2-sensitivity Δ and noise scale σ is ρ-zCDP with $\rho = \Delta^2/(2\sigma^2)$, so $\sigma = \Delta/\sqrt{2\rho}$ for a target ρ. Equation (1) is not the tightest known conversion—f-DP and Gaussian DP sharpen composition for Gaussian mechanisms [14]—but it is monotone, invertible and cheap, which is what an admission decision needs. Section VII-G shows that an Rényi accountant cannot calibrate a target of ε=0.02 at all on a practical order grid.

B. The Market Setting

We consider a market in which subjects contribute records to one or more platforms, and tenants buy answers over those records. A permissioned ledger records contributions, query charges and settlements, so that a tenant can verify that an answer corresponds to a committed query and that a charge was not silently dropped. Trustless trading designs already provide this for delivery [1], and provenance mechanisms extend it to derived artifacts [15]. What the ledger does not provide is a statement about a subject's remaining budget, because that depends on queries answered elsewhere and on charges still in flight.

Three properties of the setting shape the design. The subject population is large and heavy-tailed: most subjects appear in a handful of queries, a small minority in thousands, and it is that minority whose budgets bind. Tenants are numerous and mutually distrustful, and a tenant's incentive is to maximise the information it extracts per unit of budget, including under several identities. Queries arrive continuously, so the enforcement decision sits on the critical path of every answer the platform sells and has a latency budget of microseconds to milliseconds.

C. Approximate Counting for Thresholds

A Count-Min sketch stores a $d \times w$ matrix of counters and updates, for key x and weight v, one cell per row at the positions given by d independent hash functions [16]. The estimate is the minimum of the d cells, so collisions can only inflate it. Conservative update strengthens the guarantee:

instead of adding v to every cell it computes the current minimum m and raises each cell to at most $m+v$, which never lowers an estimate and reduces overestimation in practice [17], [16]. A space-saving summary over the same stream retains the k largest keys exactly and evicts the smallest, which is the standard way to recover heavy hitters. Recent work replaces the hash functions with a learned model [18] and revisits the row-count trade-off [16]; both optimise estimation error, whereas our requirement is weaker and, in one respect, stricter: the estimate must never fall below the truth, because it gates a safety property rather than a reported statistic.

D. Service-Level Objectives

An SLO is a predicate over a measurement window, not over a lifetime total. The distinction matters because a subject's tolerance is typically expressed as "no more than ε in any thirty days", which is a sliding window and therefore a different object from the cumulative ε that composition bounds. Enforcing only the total exhausts a budget in the first hour; enforcing only the window lets the total drift without bound. PrivFuse maintains both: a cumulative sketch that never forgets, and a ring of epoch sketches for windowed queries, with the SLO on the window and a separate hard cap on the total.

III. Threat Model and Problem Formulation

A. Entities, Trust and Adversary

A market instance comprises subjects $\mathcal{S}$ with $|\mathcal{S}| = N$, platforms $\mathcal{P}$ with $|\mathcal{P}| = n_p$, tenants $\mathcal{T}$, and an append-only ledger $\mathcal{L}$. Each subject s contributes records to one or more platforms and carries a declared cap E_s on cumulative privacy loss, together with an SLO that says how the cap may be spent over time. Each tenant submits queries naming a predicate over the contributed records and the platforms on which it is to be evaluated; the receiving platform prices the query, obtains an admission decision, evaluates the noisy answer, and settles the charge.

The coordinator is *crash-faulty but not Byzantine*: it may stop and lose volatile state, but it does not fabricate charges. The ledger is append-only and authenticated; anchoring it on a public chain strengthens non-repudiation without changing any argument here. Platform code is assumed to compute the declared mechanism, with attestation left to orthogonal infrastructure [1], [15]. Out of scope are collusion between a platform and a subject, timing and cache side channels in the query engine, and a platform that answers with less noise than it charges for; the last is a verifiable-computation problem, not an accounting one.

The adversary is a tenant controlling $k \geq 1$ accounts, possibly over several platforms, choosing queries adaptively as a function of the answers received, and aiming to extract more about a target subject than its cap allows. Because the ledger charges accounts rather than subjects, its natural strategy is *splitting*: divide a sequence that together exceeds E_s into k fragments, each individually within the per-account allowance, and issue the fragments under distinct identities. Splitting is what a rational tenant does when the price of a query is sub-linear in the information it yields; our traces sweep k from 12 to 60. The adversary may also crash the coordinator. Charging before execution and crashing loses the charge; charging after execution and crashing delivers the answer for free. Either direction breaks the accounting without any cryptographic failure, which is why settlement is part of the privacy mechanism rather than a billing detail. The adversary may inspect the ledger, replay transactions and race submissions.

B. Cost Model

Every query q carries a cost vector $\mathbf{c}(q) = (\varepsilon_{\text{pure}}, \rho, \delta)$ describing the loss charged to *each* subject in its support. For a Gaussian mechanism with sensitivity Δ and scale σ, $\rho = \Delta^2/(2\sigma^2)$ and $\varepsilon_{\text{pure}} = 0$; for a Laplace mechanism with scale b, $\varepsilon_{\text{pure}} = \Delta/b$ and $\rho = 0$. Charges add in ρ under zCDP composition and in $\varepsilon_{\text{pure}}$ under pure DP composition, so the cost of a transcript is the vector sum of its queries. Write $\sigma_s(t)$ for the cumulative vector charged to s and

$$\sigma_s(t, W) = \sum_{q\,:\, s \in \text{supp}(q),\ t_q \in (t-W,\, t]} \mathbf{c}(q) \tag{2}$$

for the spend inside a window of length W. The hard cap constrains the cumulative vector; the SLO constrains the windowed one.

C. The Enforcement Problem

An enforcement mechanism is a function $a : (q, t) \mapsto \{\text{ADMIT}, \text{THROTTLE}, \text{REJECT}\}$ evaluated before q executes. Four properties are required.

Soundness. If $a(q,t) = \text{ADMIT}$ then for every $s \in \text{supp}(q)$, $\sigma_s(t) + \mathbf{c}(q) \preceq E_s$ componentwise, and no admitted transcript ever produces $\sigma_s(t) \succ E_s$. Soundness is what distinguishes enforcement from auditing: it is a statement about the transcript that is never released, not about a sample of past releases.

Availability. Let A be the admitted fraction and U the total value of admitted answers. A mechanism should maximise A and U subject to soundness, and in particular should not refuse a query that is provably safe.

Stability. Let τ be the number of mode transitions per epoch. Oscillation between admitting and refusing is a cost not captured by A, because a tenant that cannot predict admission cannot plan a workload; we require τ to be bounded as a function of the hysteresis and dwell parameters.

Atomicity. For every execution, including those with crashes and concurrent submissions, the charge committed to a subject equals the sum of costs of the queries admitted for that subject, each query is charged at most once, and no in-flight reservation is counted as available capacity.

The SLO specification is $(\theta_{\text{soft}}, p, W)$ with $\theta_{\text{soft}} = \alpha E_s$ for a soft ratio $\alpha \in (0, 1)$: the fraction of subject-epochs whose windowed spend exceeds θ_{soft} must not exceed p, and no subject-epoch may exceed E_s. The two thresholds are deliberately separate. The soft threshold is a target the breaker may miss under adversarial load and p bounds how often; the hard cap is never a target, and one violation is a failure of the mechanism rather than a degradation of service.

D. Success Criterion for an Attacker

To compare mechanisms under splitting we report the *escape ratio*

$$R = \max_{s \in \mathcal{S}} \sigma_s(\infty)/E_s, \tag{3}$$

the worst-case ratio of the loss a subject actually incurred to the cap the subject declared. $R \leq 1$ certifies soundness on the trace; $R > 1$ quantifies the leak. The ratio folds the effect of splitting, of per-account keying and of the composition rule into one quantity that is comparable across mechanisms, and it is the quantity a regulator would ask for after the fact.

IV. Design

A. Overview

PrivFuse sits between the query planner and the execution engine of every platform, and it holds two kinds of state that answer different questions. The *settlement ledger* answers "what has this subject been charged, and what is still in flight?". It is exact, it is the authority for the hard cap, and it has no time structure. The *estimator* answers "how fast is this subject's budget being consumed, relative to the window?". It has time structure, it is consulted on every request, and it may be wrong in one direction only.

The split is deliberate. Exactness is cheap where the state is one scalar per subject and expensive where it is one counter per subject per epoch: a thirty-day window at five-minute resolution needs 8,640 epochs, so an exact windowed accountant for 10^6 subjects would need 8.6×10^9 counters. We therefore make the ledger exact and the estimator one-sided, and Section V proves that the composition preserves both properties.

Figure 1 shows the request path. A query arrives with a declared sensitivity and target ε, and the pricer converts it into a cost vector. The breaker combines subject-level pressure from the estimator with tenant-level pressure from the observed soft-violation rate and selects a mode. In NORMAL mode the full cost is reserved; in THROTTLE mode the query is answered with a reduced charge, hence more noise and less utility; in REJECT mode it is refused without touching data. Reservation is two-phase against the ledger, and only a committed charge is published to the estimator, so a failed execution costs the subject nothing.

B. Pricing a Query

A query plan declares a sensitivity upper bound Δ and a target per-query ε_q at a fixed δ. Under zCDP the platform answers with a Gaussian mechanism whose scale is $\sigma = \Delta/\sqrt{2\rho_q}$, where ρ_q inverts (1) at ε_q; a Laplace mechanism is priced with $\varepsilon_{\text{pure}} = \Delta/b$ and $\rho = 0$. The resulting $\mathbf{c}(q)$ is the unit of account for every structure that follows, and the inversion is what lets a tenant ask for "ε_q=0.25 per query" and receive a price in ρ that composes without further conversion. Pricing is conservative in one respect: the pricer takes an upper bound on sensitivity rather than a point estimate, because an under-priced query is an unbounded liability while an over-priced one merely triggers the breaker early. The controller never re-prices between reservation and commit, so a crash cannot change the price of an answer already computed.

C. A One-Sided Estimator

The estimator keeps three structures. A *cumulative layer* is a pair of Count-Min sketches, one over $\varepsilon_{\text{pure}}$ and one over ρ, so that mixed pure-DP and zCDP charges compose without conversion on the hot path. An *epoch ring* is a bounded deque of epochs, each with its own pair of sketches, supplying the windowed view of (2). A *hot table* holds the H subjects with the largest accumulated charge exactly, because those are the subjects whose budgets bind.

For key s and cost c, the conservative update computes $m_j = \min_r A_j[r, h_{j,r}(s)]$ per layer and raises every cell on s's row that falls below $m_j + c_j$ to $m_j + c_j$; the estimate is the minimum over rows. The rule never decreases a cell and never increases an estimate, which is the property we need. Standard Count-Min with plain increments also over-estimates, but conservative update keeps the estimate closer to the truth for the same width by refusing to charge collisions that the minimum already accounts for [17], [16]. With $d = \lceil \ln(1/\delta_s) \rceil$ rows and $w = \lceil e/\varepsilon_s \rceil$ columns, the estimate exceeds the true charge by at most ε_s times the layer's total mass with probability at least $1-\delta_s$, and the bound is one-sided: it bounds the excess from above rather than bracketing an unbiased estimate.

The hot table interacts with the sketch in a way that needs care. An evicted subject that returns must not restart from zero, because the sketch already holds its past. We *seed* it with the sketch's current upper bound and then accumulate exactly. The estimate is therefore the sketch's bound on the past plus the exact charge since re-admission, which still bounds the true total, with excess at most the sketch's error at the moment of eviction. Seeding from below, for instance with the point estimate minus one standard deviation, would restore accuracy at the cost of soundness, and we do not do it.

The epoch ring is sized so that a window is always covered. With epochs of length ℓ and a ring of R epochs, a window straddling a boundary spans at most $\lceil W/\ell \rceil + 1$ epochs, so we require $R \geq \lceil W/\ell \rceil + 1$ and set the ring size to exactly that bound. An earlier version used $R = \lceil W/\ell \rceil$, which silently dropped the oldest epoch of every straddling window and made the windowed estimate an under-estimate by up to one epoch of traffic. The fix is one line, but it is the difference between a one-sided estimator and one that is not, and it is the kind of error a purely cumulative test suite does not catch. Summing per-epoch bounds is itself a bound, so the ring preserves one-sidedness without further argument.

Total state is $O(dw)$ for the cumulative layer, $O(Rdw)$ for the ring and $O(H)$ for the hot table, none of which depends on N. The default uses $w = 2^{14}$, $d = 4$, $R = 3$ and $H = 1024$, occupying 4.7 MB at 10^6 subjects and not growing when the population doubles. The price is an over-estimate that is invisible for heavy hitters—the hot table holds them exactly—and bounded by ε_s times the layer mass for the tail, where

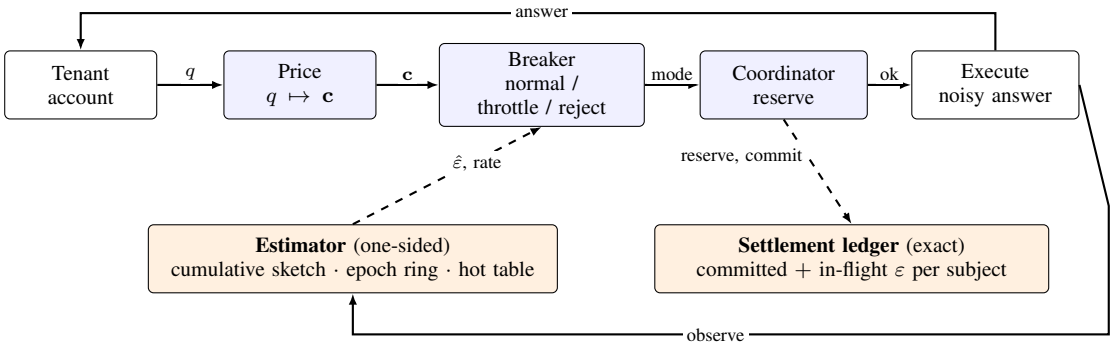


Fig. 1. Request path. Solid arrows carry requests and answers, dashed arrows carry accounting state. The ledger enforces the cumulative cap exactly; the estimator supplies windowed and tenant-level pressure and may over-estimate but never under-estimate.

the cap is least likely to bind. Section VII-B measures both effects.Section VII-B measures both effects.

D. From a Cap to an SLO

A cumulative cap alone produces the wrong behaviour at both ends. A tenant that spends a subject's budget in the first hour gets full service for an hour and nothing thereafter; a tenant that spreads the same spend over thirty days gets uniform service. Neither is what the subject agreed to, and neither is predictable for the tenant. PrivFuse therefore evaluates a population-level SLO: over a window of length W, the fraction of monitored subjects whose spend exceeds $\theta_{\text{soft}} = \alpha E$ must not exceed p, and no subject may exceed E. The default is $\alpha = 0.8$, $p = 0.01$, $W = 600\,\text{s}$ and $\ell = 300\,\text{s}$ epochs, so the ring holds three epochs.

The monitor reads the windowed estimates of the hot table rather than of the whole population, on the argument that a subject outside the hot set has, by construction, accumulated less than the eviction threshold and cannot be the subject whose spend violates the soft threshold. This makes the monitor's cost independent of N and concentrates attention where violations are possible, but it also means the reported rate is over the *violation-capable* subpopulation rather than over all subjects. The tenant-level rate is refreshed every 64 evaluations, which bounds staleness at the cost of a lag between a change in behaviour and the breaker's response.

The breaker is a four-level machine (normal, throttle, reject, freeze) that escalates immediately and de-escalates one level at a time. De-escalation requires pressure to fall below the current threshold by a margin h and remain there for d consecutive evaluations; the margin prevents oscillation when arrivals hover around a threshold, and the dwell prevents a single quiet evaluation from releasing a subject about to be charged again. In throttle mode the charge is scaled by $\beta = 0.5$, which corresponds to inflating the noise by $1/\sqrt{\beta}$ and reduces utility; the tenant receives a lower-quality answer at a lower price rather than a refusal. This converts a hard wall into a degradation curve, and it is sound because the mechanism actually executed is the one charged for.

E. Two-Phase Settlement

Reservation is all-or-nothing over the query's subject set. Under a guard that covers exactly the stripes of those subjects, the coordinator reads $\text{remaining}(s) = E_s - (\text{committed}(s) + \text{in-flight}(s))$ for every s in the support and rejects the transaction if any subject lacks room; otherwise it records a reservation with a time-to-live. The check and the record happen under the same guard, so no other transaction can observe the budget between them. A rejection is therefore not a partial charge: either the query is reserved against every subject it touches, or it touches no budget at all.

Commit is idempotent in the transaction identifier. A repeated commit for an already-committed transaction increments a duplicate counter and returns the existing record rather than charging again, which makes client retries safe. Release returns the reserved budget and is a no-op for a transaction already committed or released. Reservations neither committed nor released within their TTL are expired by a periodic tick, bounding the budget a crashed client can hold hostage to one TTL; the cost is a window in which budget is unavailable although no answer was delivered, so the TTL trades availability under crash against availability under load.

Concurrency is handled by striping. Subjects map to $L = 64$ stripes by a hash of the identifier, and a transaction acquires its subjects' stripes in sorted index order, which rules out deadlock and lets transactions over disjoint subjects proceed in parallel. A transaction spanning many subjects serialises against everything sharing its stripes—correct for a multi-subject query, and the reason our concurrency experiment shows no throughput win on a two-core host (Section VII-I). Striping is a scalability argument, not a measured speedup here.

V. Analysis

We state the properties the design guarantees and sketch why they hold. The proofs are short because the mechanisms are chosen to make them short: one-sidedness is an invariant of an update rule, safety of a check-and-record, atomicity of an identifier.

Proposition 1 (One-sidedness)**.** *For every subject s and time t, the cumulative estimate satisfies $\hat{\sigma}_s(t) \succeq \sigma_s(t)$, and the windowed estimate $\hat{\sigma}_s(t, W)$ satisfies $\hat{\sigma}_s(t, W) \succeq \sigma_s(t, W)$, where the comparison is componentwise on $(\varepsilon_{\text{pure}}, \rho)$.*

Proof: Fix a layer j and let $T_j(x)$ be the true charge of key x. We maintain the invariant that for every key x, $\min_r A_j[r, h_{j,r}(x)] \geq T_j(x)$. It holds initially with all cells zero. On an update of key x by c, the conservative rule sets each cell on x's row to $\max(a,\, m + c)$ where $m \geq T_j(x)$ is the current row minimum, so every cell on that row becomes at least $T_j(x) + c$, and cells of other keys only increase. The invariant is preserved. A subject held in the hot table is seeded with the sketch's bound at re-admission, hence at least $T_j(x)$ at that instant, and is then incremented exactly, so the invariant holds there too. The windowed estimate sums whole epochs that cover the window, each bounded below by its true charge, and charges are non-negative; hence the sum bounds the true windowed charge. ∎

Over-estimation costs availability; Section VII-B measures 0.02% relative error for the top percentile and no false negatives in 24 configurations.

Proposition 2 (Cap safety)**.** *If every platform reserves through the shared coordinator before executing, then no admitted transcript produces $\sigma_s(t) \succ E_s$ for any subject, even when tenants split their traffic across accounts and platforms.*

Proof: Let $C_s(t)$ be the committed charge and $F_s(t)$ the reserved-but-uncommitted charge at time t. The coordinator admits only when $C_s(t) + F_s(t) + \mathbf{c}(q) \preceq E_s$ for every subject in the support, and the check and the insertion occur under the same stripe guard, so no transaction interleaves between them. On commit F_s decreases and C_s increases by the same vector; on release F_s decreases. Hence $C_s(t) + F_s(t)$ is non-decreasing and bounded by E_s, and the loss actually incurred is bounded by it. Splitting does not enter the argument because the key of every reservation is the subject, not the account or the platform. ∎

Proposition 3 (Exactly-once settlement)**.** *Under crashes and retries, each admitted query is charged at most once, and no answer is delivered without a committed charge.*

Proof: Commit is keyed by the transaction identifier and returns the existing record if the transaction is already committed, so a retry cannot charge twice. The platform calls commit before returning the answer to the tenant, so an answer that leaves the platform has a committed charge; a crash before commit means the answer is never delivered. A reservation that is neither committed nor released is expired after its TTL, which bounds the budget held by a dead client. The protocol can therefore over-charge a subject whose answer was computed but lost to a crash, which is the safe direction, but cannot release data without charging for it. ∎

Proposition 4 (Bounded instability)**.** *For a single breaker key, the number of de-escalations in T evaluations is at most $\lfloor T/d \rfloor$, where d is the dwell parameter.*

Proof: A de-escalation requires d consecutive evaluations below the current threshold minus the hysteresis margin, and the counter resets on any evaluation that is not below it. Each de-escalation therefore consumes at least d distinct evaluations. ∎

The bound is loose—E7 observes 12 de-escalations over roughly 5,000 evaluations with $d = 3$—but it has the right shape: transitions are controlled by a parameter rather than by the load, and removing hysteresis raises the count to 26.

Two properties are deliberately absent. We do not prove that estimator error stays below a constant as the population grows, because the Count-Min guarantee is relative to total mass; we measure it instead. Nor do we prove that the SLO is met under every arrival process: the breaker is a feedback controller, and an abrupt load increase can drive pressure past the reject threshold before it reacts. What we prove is that the hard cap survives such a transient, because an exact structure enforces it.

VI. Implementation

PrivFuse is a reference implementation in Python 3.11: about 4,200 lines across 44 modules with 37 unit tests. The enforcement core has no dependency outside the standard library; NumPy, pandas and matplotlib are used only by the experiment drivers, so the measured per-request cost is not an artefact of a numerical stack. The package separates cost algebra, the sketch layer, the accountant, the SLO predicate and breaker, the ledger and coordinator, the platform harness and one module per experiment.

Two components are stubs. The ledger backend has an in-memory implementation and a Hyperledger Fabric adapter that raises `NotImplementedError`; the coordinator's safety argument does not depend on the backend, which sees only reserve, commit and release, but we do not measure Fabric latency and our throughput numbers are a lower bound on the settlement path. The attestation interface ships with a null attester, since verifying that a platform executed the mechanism it charged for needs remote attestation or verifiable computation. Every random choice is seeded and each experiment writes raw rows to CSV with a metadata file, so a single command regenerates every table and figure here.

VII. Evaluation

A. Methodology

We run on a two-vCPU container with 4 GiB of RAM, CPython 3.11, and a single process. The host is deliberately small: it removes any temptation to attribute a result to parallelism and makes the absolute latencies a conservative upper bound. Every experiment is repeated on three seeds and we report the mean; per-seed rows are committed with the code.

The generator produces 12,000 requests from 40 tenants over 1,500 subjects and three platforms, with subject popularity drawn from a truncated Zipf law of exponent 1.15 over ranks and 15% of requests touching more than one subject. Each subject carries a cap of ε=1.0 at $\delta = 10^{-5}$. The splitting

attack adds an adversary controlling 12 target subjects, divides a fixed budget of 0.30ε per target into fragments, and issues them under distinct tenant identities across platforms; we sweep the fragment count from 12 to 60. Identities are drawn by inverse transform from the truncated law rather than by a power-law draw modulo the population, because the latter maps large ranks back onto small ones and removes exactly the heavy hitters that make accounting hard.

We compare against five policies: *static quota*, a fixed per-tenant allowance that does not measure spend; *token bucket*, a per-tenant rate limit; *audit only*, which admits everything and flags subjects afterwards and is the deployed status quo; *exact blocking*, which keeps exact counters keyed by (tenant, subject) and refuses when that pair is exhausted; and *exact global*, which keeps exact per-subject counters shared across tenants and platforms but has no window, no breaker and no degradation. PrivFuse is the only policy that is subject-keyed, cross-platform, time-structured and memory-bounded in its estimator.

B. Is the Estimator Sound and Accurate?

The first question is whether approximation ever produces a false negative, because one false negative at admission is a cap violation. Across 24 configurations—four widths, with and without the hot table, three seeds—we observe *zero* false negatives. The property is not statistical: it follows from Proposition 1, and the experiment confirms that the implementation respects it, including on the eviction and re-admission path that the seeding rule exists to handle.

Accuracy separates along the population. For the top percentile of subjects by spend—the ones whose budgets bind—relative error is 4.3% without a hot table at width 512, 0.10% with one, and 0.02% at width 2048 and above, where the hot table rather than the width is doing the work. For the tail, absolute error is the meaningful quantity: mean absolute error is 0.071ε at width 512 and 0.013 at width 2048, while p99 absolute error falls from 0.37 to 0.10. At width 32,768 the measured error is exactly zero on this trace, because the sketch is wider than the number of distinct keys; that configuration costs 2.1 MB and is not the one we deploy. The reading is that the hot table makes the estimator usable for enforcement and the sketch makes it cheap: without hot tracking a deployment would need width 32,768 for the same heavy-hitter accuracy and would still over-estimate the tail by 0.007ε on average.

C. What Does It Cost?

The estimator costs 27–32 μs per request across widths from 2^{10} to 2^{16}, against 4.6 μs for an exact table on the same trace. The step from 2^{10} to 2^{14} is 27.3 to 32.1 μs, about 18% for a sixteen-fold increase in width, as expected from eight memory accesses per update plus the hot-table lookup. At a query latency of milliseconds—the realistic range for a DP aggregation over a sharded store—the accountant is 1–3% of the budget, and the exact-table comparison shows that the approximation is not the expensive part of enforcement; the breaker and the coordinator are.

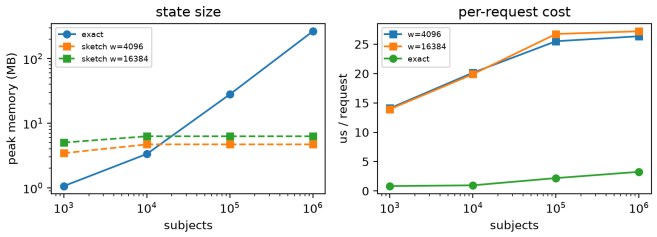


Fig. 2. Accountant state and per-request cost as the subject population grows (E8). The exact accountant grows linearly with N; the sketch is flat in N and pays a modest per-key locality penalty. Zero under-estimates at every population.

Memory tells the opposite story at small scale. For 5,000 subjects the sketch occupies 5.57 MB at width 2^{14} while the exact table occupies 1.28 MB, so the sketch is 4.3 times *larger*. The crossover is around 2×10^4 subjects, and only above it does the asymptotic argument bite. Figure 2 measures the regime that matters: from 10^3 to 10^6 subjects the exact accountant grows from 1.1 MB to 264.6 MB, while the sketch stays at 4.70 MB (width 2^{12}) and 6.27 MB (width 2^{14}). At 10^6 that is a factor of 42 to 56, and the gap widens linearly. Per-request cost grows more than we expected, from 14 μs at 10^3 subjects to 27 μs at 10^6, because a larger key space means more hot-table entries and worse locality; the estimate remains one-sided throughout, with maximum absolute error 0.057ε and worst hot-subject relative error 4.3%.

This shows that the structure answering windowed queries and computing pressure has memory independent of N, and that a naive windowed accountant—one cumulative structure per epoch—would multiply the measured 264.6 MB by the ring depth. It does not show that the whole system is $O(1)$ in N: the ledger keeps one scalar balance per subject that has ever transacted, which settlement requires, is far smaller per subject than a cost vector, and would be sharded in a deployment.

D. Does Enforcement Hold?

Table I reports the central comparison at six cap levels, each averaged over three seeds. Two rows of the table matter. At a cap of ε=1.0, PrivFuse admits 0.933 of requests and produces no hard-cap violation, while exact per-tenant blocking admits 0.978 and lets 12.7 subjects exceed the cap, the worst of them reaching 1.69ε. Static quota behaves similarly at 0.956 availability and the same violation count. The baselines' counters are exact; the violations are caused by the key. A subject charged against forty tenant budgets can accumulate forty times the cap, and on this trace natural traffic is enough to push a dozen subjects past it without any adversary.

The availability gap is the price of soundness and it is not constant. At a cap of 0.5ε PrivFuse admits 0.754 against 0.889 for exact blocking, a gap of 13.5 points; at 0.75ε the gap is 10.0 points; at 1.0ε it is 4.5 points; and at 1.5ε and above PrivFuse reaches full availability while the baselines continue to violate. The gap shrinks as the cap loosens because the breaker's soft threshold, at $0.8E$, stops binding: when the

TABLE I
ENFORCEMENT ACROSS CAP LEVELS (E3, MEAN OF THREE SEEDS). "VIOL." IS THE NUMBER OF SUBJECTS WHOSE CUMULATIVE SPEND EXCEEDED THE CAP; AVAILABILITY IS THE ADMITTED FRACTION.

	PrivFuse		Exact blocking		Static quota	
Cap	Avail.	Viol.	Avail.	Viol.	Avail.	Viol.
0.50	0.754	0	0.889	15.0	0.903	15.0
0.75	0.856	0	0.956	13.0	0.939	13.0
1.00	0.933	0	0.978	12.7	0.956	12.7
1.50	0.966	0	1.000	12.0	1.000	12.0
2.00	1.000	0	1.000	12.0	1.000	12.0
3.00	1.000	0	1.000	0.0	1.000	0.0

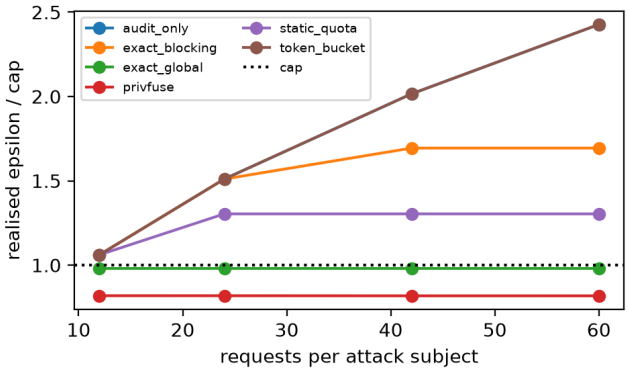


Fig. 3. Escape ratio under a splitting tenant (E4, three seeds). Policies without a subject key grow linearly in the number of fragments; subject-keyed policies are flat, and only those keep the ratio at or below one.

natural traffic never approaches the soft threshold, PrivFuse behaves like exact global accounting. The cap of 3.0ε is a control: no policy violates it, because the trace's natural maximum is 2.01ε, which confirms that the baseline violations at tighter caps are a consequence of keying rather than an artefact of the accounting implementation.

Figure 3 turns to the adversary. Under a tenant that splits a fixed budget across k identities, the escape ratio of audit-only and token-bucket policies rises linearly with k, from 1.07 at k=12 to 2.43 at k=60, because neither policy has a subject key and both therefore price each fragment against a fresh allowance. Static quota saturates at 1.30, which is the ratio of the number of tenants that can touch a subject to the cap; exact per-tenant blocking rises to 1.69 and then flattens for the same reason. PrivFuse is flat at 0.82 across the whole range, and exact global is flat at 0.98, because neither can be made to forget that the fragments concern the same subject. The flatness, not the level, is the result: the attack's only lever is the identity it uses, and identity is not part of the key.

The comparison with exact global deserves scrutiny, because it is the strongest baseline and it is also sound. At k=42 it admits 0.929 against 0.923 for PrivFuse, and it leaves 2% of the cap unused against 18%. The 18% is the residue of the soft threshold: PrivFuse throttles from $0.8E$ and throttled queries consume half a charge, so the worst subject ends below its cap rather than at it. What exact global lacks is a window, a degradation path and an estimator whose memory does not grow with N. We therefore read Figure 3 as showing that subject keying is necessary and that PrivFuse achieves it without the per-tenant escape ratios, not that PrivFuse dominates exact global on every axis.

E. Where Is the SLO Frontier?

An SLO has two knobs, the soft threshold αE and the tolerated violation rate p, and the interesting question is which pairs are achievable. Sweeping α from 0.55 to 0.95 at four values of p, holding everything else at the default, moves availability from 0.749 to 0.794 and raises the fraction of (α, p) settings at which the SLO is satisfied from 0.25 to 1.0. The hard-cap violation count is zero at every setting, including the tightest: tightening the soft target degrades throughput and never compromises the cap.

The two knobs are not independent. At α=0.55 the monitored violation rate is 2.2%, so the SLO is met only if the operator accepts $p \geq 5\%$; at α=0.85 and above the monitored rate is at most 0.22% and the default p=1% is met. This gives operators a menu—a tight soft target with a loose violation rate, or a looser target with a strict rate, at a cost of about 4.5 points of availability—but not a tight target with a strict rate, which no policy can deliver on a heavy-tailed workload: a small set of subjects crosses any threshold near zero regardless of admission policy.

The sensitivity sweep varies one factor at a time around the default, with a single seed per point. Two points are controls. With a single platform, static quota becomes sound (zero violations, worst spend 0.969ε), because a per-tenant quota on the only platform that exists is also a per-subject quota; with six platforms its worst spend rises to 1.859ε while PrivFuse stays at 0.825ε. These two points confirm that the baselines fail through aggregation rather than through an implementation defect. A heavier tail (α_{zipf}=1.5) costs 15 points of availability, from 0.936 to 0.783, because more traffic concentrates on the subjects whose budgets bind; 200 tenants cost 4 points, as more of them touch each subject between refreshes; shrinking the hot table to 64 entries costs 2 points; and moving δ from 10^{-6} to 10^{-4} raises availability from 0.920 to 0.941, since a larger δ converts the same ρ to a smaller ε. Crashes at probability 0.05 cost 1 point and produce no violations. Across all 19 points the violation count is zero and the estimator never under-estimates.

F. Ablation and Stability

Removing one component at a time (three seeds) gives the following. The full system admits 0.923 of requests at 4.77 MB and 100.9 μs per request; exact global admits 0.929 at 0.24 MB and 10.8 μs. Removing degradation, so that the breaker can only admit or refuse, costs 4.2 points of availability (0.881 against 0.923); degradation is doing the work of converting would-be refusals into answers. Shrinking the sketch from 2^{14} to 2^{9} columns changes availability not at all and cuts estimator

memory from 4.77 MB to 0.71 MB, while widening it to 2^{16} raises memory to 17.4 MB and again changes availability not at all. Width is therefore not the lever at this scale: on a 1,500-subject trace the hot table holds every subject that matters and the columns are occupied by the tail, consistent with the 0.10% heavy-hitter error at width 512 with a hot table.

The hot-table row is a warning. Disabling the table raises availability from 0.923 to 0.930 and lowers cost from 101 to 80 μs, which looks like a free win until one notices that the breaker's flap count falls to zero. The tenant-level monitor samples its violation rate from the hot table, so a capacity of one leaves the monitor with almost nothing to sample and the SLO machinery goes blind. The row is not evidence that the hot table hurts: it is evidence that the monitor and the estimator share a data structure, so ablating one silently ablates the other.

The last row is the honest comparison. At the trace's scale, exact global accounting uses less memory and is nine times faster per request, because 1,500 subjects fit comfortably in a hash table. Its disadvantage appears at 10^6 subjects (Figure 2), where the table is 265 MB, and in the windowed SLO, which it cannot express at all. A small deployment with a cumulative-only policy would be better served by exact counting; PrivFuse is the right structure when the population is large or the SLO is windowed.

Stability is measured on a load oscillating around the throttle threshold: with hysteresis and dwell the breaker makes zero transitions, and with both disabled it makes 26, of which 13 are de-escalations followed by immediate re-escalation.

G. Why zCDP and Not Rényi DP

The choice of composition algebra is usually a matter of tightness. For admission control it is a matter of solvability. An Rényi accountant evaluates a discrete grid of orders and converts to (ε, δ); the conversion has a floor at $\log(1/\delta)/(\alpha_{\max} - 1)$, which for $\delta = 10^{-5}$ and $\alpha_{\max} = 512$ is 0.0225. A tenant asking for ε_q=0.02 is therefore below the floor, and no finite noise scale certifies the target. Our Rényi implementation responds by saturating at its noise cap of $\sigma = 1000$, 4.17 times the scale zCDP computes, and reports a per-charge ε of 0.023 with `reached_target` false rather than silently accepting the query. Basic composition, which sums ε, needs $\sigma = 240$ for the same target but reports 0.2008 after 100 charges, ten times the budget zCDP reports.

At ε_q=0.1 the three backends agree to within 0.004% in σ and in the composed ε (1.0195 for zCDP, 1.0194 for Rényi); at ε_q=0.5 they agree to within 0.006%. The conclusion is narrow: for the per-query budgets a market actually prices—a few hundredths to a few tenths of an ε—the closed-form zCDP conversion is the only one of the three that can both calibrate the mechanism and compose it without discarding an order of magnitude. Rényi DP remains the right tool for training-time accounting at larger budgets, where its constants win; it is the wrong tool for admission at ε_q=0.02.

H. What Soundness Costs in Utility

The utility experiment asks a subject's data to support queries until the cap is exhausted, and measures the accuracy of the resulting answer. At ε_q=1.0 PrivFuse answers one query and reaches 0.823 accuracy, against 0.900 for an unbounded policy that answers 100 queries and spends 100ε; at 0.5ε it answers two at 0.552 against 0.881; at 0.25ε it answers four at 0.542 against 0.871. The unbounded and static-quota policies each exceed the cap on one subject in every seed, so their accuracy is bought with a violation.

The gap at small ε_q is the budget, not overhead: 1.0ε at 0.25ε per query funds four queries, and four noisy answers cannot match a hundred. Among mechanisms that respect the cap, PrivFuse is the only one in our set that answers more than zero queries at every ε_q with zero violations. The loss from degradation is smaller: 699 of 5,088 admitted queries are throttled at 70% of nominal value, the 4% gap to exact global.

I. Settlement and Concurrency

The settlement invariants are checked directly: a repeated commit charges once, a timeout releases the reservation, a repeated reserve returns the same reservation and cannot be re-bound to another tenant, a crash between reserve and commit leaves no double spend, and an expired reservation is recovered. All five pass. We report them because the failure they prevent is invisible in aggregate statistics: a lost charge does not change availability or utility, it only makes the next query admissible.

The concurrency experiment is where our results are negative. In the atomicity test, 16 threads issue 3,200 attempts against a subject whose cap funds exactly 100 charges. Every thread count from 2 to 16 admits exactly 100, spends exactly 1.0ε, and records zero violations, so check-and-reserve is atomic in practice and not only in argument. Throughput does not follow. Striped locking reaches 71.9k operations per second on one thread, 67.3k on two and 49.6k on four, while a single global lock reaches 99.6k on one thread and 51.0k on four. Striping costs 28% of single-thread throughput, because acquiring several locks is dearer than acquiring one, and it does not recover the loss at four threads on a two-core host, where the global lock is not the bottleneck and CPython's interpreter lock is. We expected the striped design to win at four threads and it does not; the fair statement is that striping removes a serialisation point that would bind on a many-core host with a blocking storage backend.

Two caveats follow. The coordinator is a single process, so the numbers bound the protocol rather than a distributed deployment; a Fabric-backed ledger adds a round trip per commit, which we did not measure. And the throughput test uses one subject, maximising contention; a workload over disjoint subjects would exercise the stripes as intended.

VIII. Related Work

Privacy accounting. Composition has spent two decades getting tighter, and the trajectory is visible in recent work on continual observation, which separates oblivious from adaptive adversaries [19] and revisits the cost of answering online queries [20]. Tight auditing has become its own line,

from reconstruction attacks on production query systems [8] to Gaussian-DP audits of synthetic data generators [9] and analyses of the gap between provable and auditable privacy in DP-SGD [10]. All of it asks how large the loss was; PrivFuse asks whether the next release may happen, which has a different error budget. We do not improve a composition bound. We take a standard one, zCDP [5], and make it cheap and monotone enough to sit on an admission path, alongside alternatives such as Rényi [4] and Gaussian DP [14] whose relative merits we measure for that purpose (Section VII-G).

Systems that enforce privacy. PINQ established the pattern of a query interface that charges a budget per analyst [21], and production libraries apply it to training [22]. These systems enforce a budget per *analyst* and assume the analyst is the unit of privacy. In a market the unit is the subject and the analyst is an adversary who can be several analysts, which is why the key and the cross-tenant aggregation matter more than the query language. Our settlement protocol has no counterpart in this line; the closest analogue is accounting in a database transaction.

Auditing and governance. A parallel literature builds the machinery for post-market monitoring [11] and ethics-based audit [12], and empirical studies of organisational practice [7] show how far declared policy is from measured behaviour. Privacy-policy formalisms give declarations a semantics [6], and recent work extends governance to AI-mediated platforms [23]. PrivFuse is complementary: it assumes the declaration exists and makes it enforceable at the point of sale, and an auditor who can read a ledger of admissions need not infer what happened from the released artifacts.

Data markets. Trustless trading has occupied the blockchain systems community, with recent designs scaling delivery and settlement [1] and provenance mechanisms tracking derived artifacts [15]. These answer "did I receive what I paid for?". The privacy question—"may this answer be sold at all?"—is orthogonal and, in our reading of deployments, unaddressed: the ledger records payments, not budgets. We keep the ledger as the settlement substrate and add a budget root that every platform consults, the smallest change that lets the same infrastructure answer both questions.

Approximate counting. Count-Min sketches and their conservative variants are well understood [16], [17], including on power-law streams and for external storage [24], and learned variants reduce error at the same width [18]. Our use is unfashionable: we want not the smallest error but an error with a known sign, which rules out unbiased estimators and makes the conservative update the right primitive rather than an optimisation.

Heterogeneous budgets and mechanisms. Charging subjects different budgets is a recognised generalisation [25], and per-subject constraints appear in genomic query answering [26] and distance-preserving transformations [27]; these establish that per-subject accounting is meaningful, not how to maintain it across tenants at market scale. The answers PrivFuse prices come from standard mechanisms, from DP synthesis [28] to spectral perturbation [29], geometric preconditioning [30], [31], federated training [32] and private sensing [33]. We assume only that each query exposes a cost vector, as all of these do in some form; calibrating to data [34], releasing under a sensitivity budget [35] or handling high-sensitivity queries through concentration [36] changes the vector, not the enforcement layer.

IX. Discussion

Three principles seem to us more transferable than the specific parameters. Split the accounting state by the kind of question it answers: exact where the quantity is money and must be settled, one-sided where it has time structure and must be cheap. Key every charge on the party that bears the loss rather than the party that pays, because the gap between the two is the gap an adversary exploits. And treat the charge as a transaction with a commit point, because a counter that can be incremented without one will eventually be incremented twice or not at all.

The deployment path we have in mind replicates the coordinator across a small set of consensus nodes, owns each subject on one node so the critical section stays local, and shards the ledger by subject range. That removes the single-process bottleneck but introduces a trust question we have not solved: a subject's budget now lives on one node, and a malicious node could under-report a charge. Signed receipts make under-reporting detectable after the fact, which is weaker than the guarantee inside one process; closing the gap needs Byzantine agreement or a proof that the charge corresponds to the mechanism executed.

A less obvious problem is that the accounting layer is itself sensitive. The ledger records which subjects appear in which queries, and the hot table is close to a list of the subjects about whom the most has been asked, so a reader of the accounting state learns participation patterns without learning any answer. We have not addressed this; accumulators, oblivious access, or an enclave for the hot table each cost something the current design does not budget for, and in our view this is the most important open problem here.

Two further limitations bound the claims. The SLO monitor samples the hot table, so its coverage depends on the hot capacity, and sampling the epoch ring directly is a fix we have not evaluated. And the adversary chooses its split in advance; one that observes throttling and re-times its fragments could extract more than our ratios suggest, though the key still binds it.

What would falsify the approach? If a market's population is small and its policy cumulative, exact counting is cheaper and faster, as our ablation shows at $1{,}500$ subjects; with one platform, per-tenant quotas are already sound. If a query's cost cannot be predicted before execution—because its sensitivity depends on the data—admission must price conservatively or fall back to post-hoc charging, and the guarantee weakens to the one the auditing literature provides. The design is for the case that remains: many subjects, several platforms, many tenants, and a promise that must hold before the answer leaves the building.

X. Conclusion

Privacy budgets in data markets are enforced after the fact, keyed on the party that pays, and reconciled outside the transaction that spends them. Each choice is individually reasonable and together they fail: the wrong key lets a tenant split its traffic, the wrong timing releases the answer before the overspend is known, and the wrong data structure loses the charge on a crash. PrivFuse moves the decision to admission around three commitments—an exact ledger where settlement requires exactness, a one-sided sketch where time makes exactness unaffordable, and a two-phase reservation where atomicity separates a budget from a guess. On a million subjects the estimator holds 4.7 MB where exact accounting needs 265 MB; enforcement admits 0.93 of requests at 1.0ε with no violations where per-tenant blocking admits 0.98 and breaches the cap a dozen times; the escape ratio under a splitting tenant stays at 0.82 where policies without a subject key reach 2.43. The cost is measurable—about 4.5 points of availability at a tight cap, 4.2 without degradation, no throughput win from striping—and we report it alongside the gains. The next step is to make the accounting state itself private, and to replace the trust we place in one coordinator with a proof that the charge matches the mechanism that ran.

References

[1] X. Liu, Z. Guo, L. Liu, M. Zheng, Y. Qiu, and L. Jiang, "Yotta: A large-scale trustless data trading scheme for blockchain system," Nanyang Blockchain Conference 2025, 2025. [Online]. Available: http://arxiv.org/abs/2506.19368v1

[2] F. Fioretto, P. V. Hentenryck, and J. Ziani, "Differential privacy overview and fundamental techniques," arXiv:2411.04710, 2024. [Online]. Available: http://arxiv.org/abs/2411.04710v1

[3] D. Desfontaines and B. Pejó, "SoK: Differential privacies," *Proceedings on Privacy Enhancing Technologies*, no. 2, 2020. [Online]. Available: http://arxiv.org/abs/1906.01337v6

[4] I. Mironov, "Rényi differential privacy," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 2017, pp. 263–275. [Online]. Available: http://arxiv.org/abs/1702.07476v3

[5] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," arXiv:1605.02065, 2016. [Online]. Available: http://arxiv.org/abs/1605.02065v1

[6] M. Majedi and K. Barker, "The privacy policy permission model: A unified view of privacy policies," *Transactions on Data Privacy*, vol. 14, no. 1, pp. 1–36, 2021. [Online]. Available: http://arxiv.org/abs/2403.17414v1

[7] A. Kapp, "Collection, usage and privacy of mobility data in the enterprise and public administrations," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 4, pp. 440–456, 2022. [Online]. Available: http://arxiv.org/abs/2407.03732v1

[8] A. Cohen and K. Nissim, "Linear program reconstruction in practice," arXiv:1810.05692, 2018. [Online]. Available: http://arxiv.org/abs/1810.05692v2

[9] G. Ganev, M. S. M. S. Annamalai, and B. Kulynych, "Tight auditing of differential privacy in MST and AIM," arXiv:2604.18352, 2026. [Online]. Available: http://arxiv.org/abs/2604.18352v1

[10] S. Modi, S. Balaji, Y. Zhu, G. Kamath, and K. Tian, "Revisiting the provable-auditable privacy gap of DP-SGD," arXiv:2608.28934, 2026. [Online]. Available: http://arxiv.org/abs/2608.28934v1

[11] J. Mökander, M. Axente, F. Casolari, and L. Floridi, "Conformity assessments and post-market monitoring: A guide to the role of auditing in the proposed European AI regulation," *Minds and Machines*, vol. 32, no. 2, pp. 241–268, 2021. [Online]. Available: http://arxiv.org/abs/2111.05071v1

[12] J. Mökander, J. Morley, M. Taddeo, and L. Floridi, "Ethics-based auditing of automated decision-making systems: Nature, scope, and limitations," *Science and Engineering Ethics*, vol. 27, no. 4, 2021. [Online]. Available: http://arxiv.org/abs/2110.10980v1

[13] S. P. Kasiviswanathan and A. Smith, "On the 'semantics' of differential privacy: A Bayesian formulation," *Journal of Privacy and Confidentiality*, vol. 6, no. 1, 2014. [Online]. Available: http://arxiv.org/abs/0803.3946v4

[14] J. Dong, A. Roth, and W. J. Su, "Rejoinder: Gaussian differential privacy," arXiv:2104.01987, 2021. [Online]. Available: http://arxiv.org/abs/2104.01987v2

[15] J. Zhu and L. Wang, "Auditing data provenance in real-world text-to-image diffusion models for privacy and copyright protection," arXiv:2506.11434, 2025. [Online]. Available: http://arxiv.org/abs/2506.11434v1

[16] É. Fusy and G. Kucherov, "Count-min sketch with variable number of hash functions: an experimental study," arXiv:2302.05245, 2023. [Online]. Available: http://arxiv.org/abs/2302.05245v2

[17] ——, "Phase transition in count approximation by Count-Min sketch with conservative updates," in *Approximation and Online Algorithms (WAOA 2022)*, ser. Lecture Notes in Computer Science. Springer, 2023, pp. 232–246. [Online]. Available: http://arxiv.org/abs/2203.15496v2

[18] K. Nishishita, A. Sato, and Y. Matsui, "Optimized learned count-min sketch," arXiv:2512.12252, 2025. [Online]. Available: http://arxiv.org/abs/2512.12252v1

[19] M. Bun, M. Gaboardi, and C. Wagaman, "Separating oblivious and adaptive differential privacy under continual observation," arXiv:2603.11029, 2026. [Online]. Available: http://arxiv.org/abs/2603.11029v2

[20] M. Bun, T. Steinke, and J. Ullman, "Make up your mind: The price of online queries in differential privacy," arXiv:1604.04618, 2016. [Online]. Available: http://arxiv.org/abs/1604.04618v1

[21] H. Ebadi and D. Sands, "Featherweight PINQ," arXiv:1505.02642, 2015. [Online]. Available: http://arxiv.org/abs/1505.02642v1

[22] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov, "Opacus: User-friendly differential privacy library in PyTorch," arXiv:2109.12298, 2021. [Online]. Available: http://arxiv.org/abs/2109.12298v4

[23] A. Shouli, A. Barthwal, M. Campbell, and A. K. Shrestha, "Ethical AI for young digital citizens: A call to action on privacy governance," *Security and Privacy*, vol. 9, no. 2, p. e70202, 2026. [Online]. Available: http://arxiv.org/abs/2503.11947v4

[24] M. Goswami, D. Medjedovic, E. Mekic, and P. Pandey, "Buffered Count-Min sketch on SSD: Theory and experiments," arXiv:1804.10673, 2018. [Online]. Available: http://arxiv.org/abs/1804.10673v1

[25] M. Alaggan, S. Gambs, and A.-M. Kermarrec, "Heterogeneous differential privacy," arXiv:1504.06998, 2015. [Online]. Available: http://arxiv.org/abs/1504.06998v1

[26] B. Jiang, M. Seif, R. Tandon, and M. Li, "Answering count queries for genomic data with perfect privacy," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 3862–3875, 2023. [Online]. Available: http://arxiv.org/abs/2202.01806v2

[27] K. Kenthapadi, A. Korolova, I. Mironov, and N. Mishra, "Privacy via the Johnson-Lindenstrauss transform," *Journal of Privacy and Confidentiality*, vol. 5, no. 1, pp. 39–71, 2013. [Online]. Available: http://arxiv.org/abs/1204.2606v1

[28] U. Saha, T. M. Tonoy, and H. Imtiaz, "DP-CDA: An algorithm for enhanced privacy preservation in dataset synthesis through randomized mixing," *Security and Privacy*, vol. 9, no. 2, p. e70207, 2026. [Online]. Available: http://arxiv.org/abs/2411.16121v3

[29] C. Feng, N. Xu, W. Wen, P. Venkitasubramaniam, and C. Ding, "Spectral-DP: Differentially private deep learning through spectral perturbation and filtering," arXiv:2307.13231, 2023. [Online]. Available: http://arxiv.org/abs/2307.13231v1

[30] M. M. V. den Bosch, R. Taiello, A. S. Aillet, A. Protani, M. A. G. Ballester, and L. Serio, "DP-KFC: Data-free preconditioning for privacy-preserving deep learning," arXiv:2605.13418, 2026. [Online]. Available: http://arxiv.org/abs/2605.13418v1

[31] J. Duan, H. Hu, Q. Ye, and X. Sun, "Technical report: Full version of analyzing and optimizing perturbation of DP-SGD geometrically," International Conference of Data Engineering (ICDE 2025), 2025. [Online]. Available: http://arxiv.org/abs/2504.05618v1

[32] M. Han, T. Zhu, and W. Zhou, "Momentum gradient descent federated learning with local differential privacy," arXiv:2209.14086, 2022. [Online]. Available: http://arxiv.org/abs/2209.14086v2

[33] Y. Beugin, Q. Burke, B. Hoak, R. Sheatsley, E. Pauley, G. Tan, S. R. Hussain, and P. McDaniel, "Building a privacy-preserving smart camera system," PoPETS (Proceedings on Privacy Enhancing Technologies Symposium) 2022, 2022. [Online]. Available: http://arxiv.org/abs/2201.09338v1

[34] D. Proserpio, S. Goldberg, and F. McSherry, "Calibrating data to sensitivity in private data analysis," *Proceedings of the VLDB Endowment*, vol. 7, no. 8, pp. 637–648, 2014.

[35] M. Hardt, K. Ligett, and F. McSherry, "A simple and practical algorithm for differentially private data release," arXiv:1012.4763, 2010. [Online]. Available: http://arxiv.org/abs/1012.4763v2

[36] K. Nissim and U. Stemmer, "Concentration bounds for high sensitivity functions through differential privacy," arXiv:1703.01970, 2017. [Online]. Available: http://arxiv.org/abs/1703.01970v1